==========

CAPSTONE PROJECT

NOTEBOOK 23: FULL GENRE INVENTORY AND MULTI-LABEL PREPARATION

==========

Purpose:

This notebook audits all genres present in the FMA metadata

and prepares the project for full multi-label genre modelling.

The goal is to:

1. Load tracks.csv and genres.csv

2. Parse genre fields from the metadata

3. Build a complete inventory of genres used in the dataset

4. Map genre hierarchy using parent-child relationships

5. Identify which genres are realistically usable for modelling

6. Prepare a large-subset multi-label master table

==========

In [1]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import ast
import numpy as np
import pandas as pd
```

In [2]:

```
# ============================================================
# 2. LOAD TRACK AND GENRE METADATA
# ============================================================

tracks = pd.read_csv(
    "../data/raw/metadata/tracks.csv",
    header=[0, 1],
    index_col=0,
    low_memory=False
)

genres = pd.read_csv(
    "../data/raw/metadata/genres.csv",
    index_col=0
)

tracks.index = tracks.index.astype(int)
genres.index = genres.index.astype(int)
genres.columns = [str(c).strip() for c in genres.columns]

print("Tracks shape:", tracks.shape)
print("Genres shape:", genres.shape)

display(tracks.head())
display(genres.head())
```

```
Tracks shape: (106574, 52)
Genres shape: (163, 4)
```

track_id	comments	date_created	date_released	engineer	favorites	id	information	listens
2	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
3	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
5	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
10	0	2008-11-26 01:45:08	2008-02-06 00:00:00	NaN	4	6	NaN	47632
20	0	2008-11-26 01:45:05	2009-01-06 00:00:00	NaN	2	4	"spiritual songs" from Nicky Cook	2710

5 rows × 52 columns

genre_id	#tracks	parent	title	top_level
1	8693	38	Avant-Garde	38
2	5271	0	International	2
3	1752	0	Blues	3
4	4126	0	Jazz	4
5	4106	0	Classical	5

In [3]:

```
# ============================================================
# 3. DEFINE HELPER FUNCTIONS
# ============================================================

def dedupe_keep_order(seq):
    seen = set()
```

```
    out = []
    for x in seq:
        if x not in seen:
            out.append(x)
            seen.add(x)
    return out

def parse_genre_field(value):
    """
    Robust parser for FMA genre fields.
    Handles values like:
    - []
    - [12, 34]
    - [{'genre_id': 12, 'genre_title': 'Rock'}, ...]
    """
    if pd.isna(value):
        return []

    if isinstance(value, list):
        parsed = value
    else:
        text = str(value).strip()
        if text in ["", "[]", "nan", "None"]:
            return []
        try:
            parsed = ast.literal_eval(text)
        except Exception:
            return []

    ids = []

    if isinstance(parsed, dict):
        parsed = [parsed]

    if isinstance(parsed, (list, tuple, set)):
        for item in parsed:
            if isinstance(item, dict):
                for key in ["genre_id", "id"]:
                    if key in item:
                        try:
                            ids.append(int(item[key]))
                            break
                        except Exception:
                            pass
            else:
                try:
                    ids.append(int(item))
                except Exception:
                    pass
    else:
        try:
            ids.append(int(parsed))
        except Exception:
            pass

    return dedupe_keep_order(ids)
```

```
def get_audio_path(track_id, base_dir="../data/raw/audio/fma_large"):
    track_id_str = f"{int(track_id):06d}"
    folder = track_id_str[:3]
    return os.path.join(base_dir, folder, f"{track_id_str}.mp3")

def assign_modelling_tier(n):
    if n >= 1000:
        return "Tier 1: Strong"
    elif n >= 250:
        return "Tier 2: Workable"
    elif n >= 100:
        return "Tier 3: Limited"
    elif n >= 25:
        return "Tier 4: Sparse"
    elif n > 0:
        return "Tier 5: Very Sparse"
    else:
        return "Unused"
```

In [4]:

```
# ============================================================
# 4. PARSE GENRE COLUMNS FROM TRACK METADATA
# ============================================================

tracks["genres_ids"] = tracks[("track", "genres")].apply(parse_genre_field)
tracks["genres_all_ids"] = tracks[("track", "genres_all")].apply(parse_genre_field)
tracks["genre_top_name"] = tracks[("track", "genre_top")]

print("Parsed genre columns added.")

preview_df = pd.DataFrame({
    "genre_top": tracks[("track", "genre_top")].head(10).values,
    "genres_ids": tracks["genres_ids"].head(10).values,
    "genres_all_ids": tracks["genres_all_ids"].head(10).values
})

display(preview_df)
```

```
Parsed genre columns added.
```

	genre_top	genres_ids	genres_all_ids
0	Hip-Hop	[21]	[21]
1	Hip-Hop	[21]	[21]
2	Hip-Hop	[21]	[21]
3	Pop	[10]	[10]
4	NaN	[76, 103]	[17, 10, 76, 103]
5	NaN	[76, 103]	[17, 10, 76, 103]
6	NaN	[76, 103]	[17, 10, 76, 103]
7	NaN	[76, 103]	[17, 10, 76, 103]
8	NaN	[76, 103]	[17, 10, 76, 103]
9	Hip-Hop	[21]	[21]

In [5]:

```
# ============================================================
# 5. IDENTIFY GENRE METADATA COLUMNS
# ============================================================

name_col = next((c for c in ["title", "genre_title", "name"] if c in
genres.columns), None)
parent_col = next((c for c in ["parent", "parent_id"] if c in genres.columns),
None)
top_level_col = next((c for c in ["top_level"] if c in genres.columns), None)

if name_col is None:
    object_cols = genres.select_dtypes(include=["object"]).columns.tolist()
    if len(object_cols) > 0:
        name_col = object_cols[0]
    else:
        raise ValueError("Could not find a usable genre name column in genres.csv")

print("Using genre name column:", name_col)
print("Using parent column:", parent_col)
print("Using top-level column:", top_level_col)
```

```
Using genre name column: title
Using parent column: parent
Using top-level column: top_level
```

In [6]:

```
# ============================================================
# 6. BUILD GENRE NAME AND HIERARCHY MAPS
# ============================================================

genre_name_map = genres[name_col].to_dict()
```

```
if parent_col is not None:
    parent_map = genres[parent_col].to_dict()
else:
    parent_map = {gid: np.nan for gid in genres.index}

def normalize_parent_id(x):
    try:
        x = int(x)
        if x == 0:
            return None
        return x
    except Exception:
        return None

def get_root_genre_id(genre_id):
    current = int(genre_id)
    seen = set()

    while True:
        parent = normalize_parent_id(parent_map.get(current, None))

        if parent is None or parent == current or parent in seen:
            return current

        seen.add(current)
        current = parent

def get_parent_name(genre_id):
    parent = normalize_parent_id(parent_map.get(genre_id, None))
    if parent is None:
        return None
    return genre_name_map.get(parent, f"genre_{parent}")
```

In [7]:

```
# ============================================================
# 7. COUNT GENRE USAGE IN FULL METADATA
# ============================================================

# Direct genres
full_direct = pd.DataFrame({
    "genre_id": tracks["genres_ids"].copy()
}).explode("genre_id")

full_direct = full_direct[full_direct["genre_id"].notna()].copy()
full_direct["genre_id"] = full_direct["genre_id"].astype(int)
full_direct_counts = full_direct["genre_id"].value_counts()

# genres_all
full_all = pd.DataFrame({
    "genre_id": tracks["genres_all_ids"].copy()
}).explode("genre_id")

full_all = full_all[full_all["genre_id"].notna()].copy()
full_all["genre_id"] = full_all["genre_id"].astype(int)
```

```
full_all_counts = full_all["genre_id"].value_counts()

print("Unique genres in direct 'genres' field:", full_direct_counts.shape[0])
print("Unique genres in 'genres_all' field:", full_all_counts.shape[0])
```

```
Unique genres in direct 'genres' field: 161
Unique genres in 'genres_all' field: 161
```

In [8]:

```
# ============================================================
# 8. BUILD LARGE-SUBSET MULTI-LABEL COUNTS
# ============================================================

large_tracks = tracks[tracks[("set", "subset")] == "large"].copy()

large_tracks["audio_path"] = [get_audio_path(idx) for idx in large_tracks.index]
large_tracks["audio_exists"] = large_tracks["audio_path"].apply(os.path.exists)

large_tracks_audio = large_tracks[large_tracks["audio_exists"] == True].copy()

large_all = pd.DataFrame({
    "genre_id": large_tracks_audio["genres_all_ids"].copy()
}).explode("genre_id")

large_all = large_all[large_all["genre_id"].notna()].copy()
large_all["genre_id"] = large_all["genre_id"].astype(int)
large_all_counts = large_all["genre_id"].value_counts()

top_level_summary = (
    pd.DataFrame({
        "all_tracks_top_level_count": tracks[("track",
"genre_top")].value_counts(dropna=True),
        "large_audio_top_level_count": large_tracks_audio[("track",
"genre_top")].value_counts(dropna=True)
    })
    .fillna(0)
    .astype(int)
    .reset_index()
    .rename(columns={"index": "genre_top"})
    .sort_values("large_audio_top_level_count", ascending=False)
)

print("Large subset with audio shape:", large_tracks_audio.shape)
print("Unique genres in large audio subset via genres_all:",
large_all_counts.shape[0])

display(top_level_summary.head(20))
```

```
Large subset with audio shape: (81574, 57)
Unique genres in large audio subset via genres_all: 161
```

	(track, genre_top)	all_tracks_top_level_count	large_audio_top_level_count
5	Experimental	10608	8357
13	Rock	14182	7079
4	Electronic	9372	3058
7	Hip-Hop	3552	1351
6	Folk	2803	1284
12	Pop	2332	1146
8	Instrumental	2079	729
1	Classical	1230	611
9	International	1389	371
15	Spoken	423	305
10	Jazz	571	187
11	Old-Time / Historic	554	44
0	Blues	110	36
14	Soul-RnB	175	21
2	Country	194	16
3	Easy Listening	24	3

In [9]:

```
# ============================================================
# 9. BUILD COMPLETE GENRE INVENTORY TABLE
# ============================================================

genre_inventory = pd.DataFrame(index=genres.index.copy())
genre_inventory.index.name = "genre_id"

genre_inventory["genre_name"] = genre_inventory.index.map(lambda x:
genre_name_map.get(x, f"genre_{x}"))
genre_inventory["parent_id"] = genre_inventory.index.map(lambda x:
normalize_parent_id(parent_map.get(x, None)))
genre_inventory["parent_name"] = genre_inventory.index.map(get_parent_name)
genre_inventory["root_genre_id"] = genre_inventory.index.map(get_root_genre_id)
genre_inventory["root_genre_name"] = genre_inventory["root_genre_id"].map(
    lambda x: genre_name_map.get(x, f"genre_{x}")
)

if top_level_col is not None:
    genre_inventory["top_level_flag"] = genres[top_level_col]
else:
    genre_inventory["top_level_flag"] = np.nan

genre_inventory["full_direct_count"] =
```

```
genre_inventory.index.map(full_direct_counts).fillna(0).astype(int)
genre_inventory["full_genres_all_count"] =
genre_inventory.index.map(full_all_counts).fillna(0).astype(int)
genre_inventory["large_audio_genres_all_count"] =
genre_inventory.index.map(large_all_counts).fillna(0).astype(int)

genre_inventory["modelling_tier"] =
genre_inventory["large_audio_genres_all_count"].apply(assign_modelling_tier)

genre_inventory = genre_inventory.sort_values(
    ["large_audio_genres_all_count", "full_genres_all_count"],
    ascending=False
)

print("Complete genre inventory:")
display(genre_inventory.head(30))
```

```
Complete genre inventory:
```

genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_fla
38	Experimental	NaN	None	38	Experimental	1
15	Electronic	NaN	None	15	Electronic	1
12	Rock	NaN	None	12	Rock	1
1235	Instrumental	NaN	None	1235	Instrumental	123
10	Pop	NaN	None	10	Pop	1
17	Folk	NaN	None	17	Folk	1
1	Avant-Garde	38.0	Experimental	38	Experimental	1
107	Ambient	1235.0	Instrumental	1235	Instrumental	123
32	Noise	38.0	Experimental	38	Experimental	1
76	Experimental Pop	10.0	Pop	10	Pop	1
21	Hip-Hop	NaN	None	21	Hip-Hop	2
25	Punk	12.0	Rock	12	Rock	1
41	Electroacoustic	38.0	Experimental	38	Experimental	1
27	Lo-Fi	12.0	Rock	12	Rock	1
18	Soundtrack	1235.0	Instrumental	1235	Instrumental	123
42	Ambient Electronic	15.0	Electronic	15	Electronic	1
2	International	NaN	None	2	International	
66	Indie-Rock	12.0	Rock	12	Rock	1
250	Improv	38.0	Experimental	38	Experimental	1
4	Jazz	NaN	None	4	Jazz	
103	Singer-Songwriter	17.0	Folk	17	Folk	1
5	Classical	NaN	None	5	Classical	
30	Field Recordings	38.0	Experimental	38	Experimental	1
247	Musique Concrete	38.0	Experimental	38	Experimental	1
236	IDM	15.0	Electronic	15	Electronic	1
47	Drone	38.0	Experimental	38	Experimental	1
183	Glitch	15.0	Electronic	15	Electronic	1

genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_fl
85	Garage	12.0	Rock	12	Rock	
33	Psych-Folk	17.0	Folk	17	Folk	
70	Industrial	12.0	Rock	12	Rock	

In [10]:

```
# ============================================================
# 10. CREATE MODELLING CANDIDATE TABLE
# ============================================================
# These are genres that appear often enough in the large audio subset
# to be considered for future multi-label modelling.

candidate_genres = genre_inventory[
    genre_inventory["large_audio_genres_all_count"] >= 25
].copy()

candidate_genres = candidate_genres.reset_index()

print("Candidate genres with at least 25 large-audio occurrences:",
candidate_genres.shape[0])
display(candidate_genres.head(30))
```

```
Candidate genres with at least 25 large-audio occurrences: 150
```

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_lev
0	38	Experimental	NaN	None	38	Experimental	
1	15	Electronic	NaN	None	15	Electronic	
2	12	Rock	NaN	None	12	Rock	
3	1235	Instrumental	NaN	None	1235	Instrumental	
4	10	Pop	NaN	None	10	Pop	
5	17	Folk	NaN	None	17	Folk	
6	1	Avant-Garde	38.0	Experimental	38	Experimental	
7	107	Ambient	1235.0	Instrumental	1235	Instrumental	
8	32	Noise	38.0	Experimental	38	Experimental	
9	76	Experimental Pop	10.0	Pop	10	Pop	
10	21	Hip-Hop	NaN	None	21	Hip-Hop	
11	25	Punk	12.0	Rock	12	Rock	
12	41	Electroacoustic	38.0	Experimental	38	Experimental	
13	27	Lo-Fi	12.0	Rock	12	Rock	
14	18	Soundtrack	1235.0	Instrumental	1235	Instrumental	
15	42	Ambient Electronic	15.0	Electronic	15	Electronic	
16	2	International	NaN	None	2	International	
17	66	Indie-Rock	12.0	Rock	12	Rock	
18	250	Improv	38.0	Experimental	38	Experimental	
19	4	Jazz	NaN	None	4	Jazz	
20	103	Singer-Songwriter	17.0	Folk	17	Folk	
21	5	Classical	NaN	None	5	Classical	
22	30	Field Recordings	38.0	Experimental	38	Experimental	
23	247	Musique Concrete	38.0	Experimental	38	Experimental	
24	236	IDM	15.0	Electronic	15	Electronic	
25	47	Drone	38.0	Experimental	38	Experimental	
26	183	Glitch	15.0	Electronic	15	Electronic	
27	85	Garage	12.0	Rock	12	Rock	

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_lev
28	33	Psych-Folk	17.0	Folk	17	Folk	
29	70	Industrial	12.0	Rock	12	Rock	

In [11]:

```
# ============================================================
# 11. CREATE LARGE-SUBSET MULTI-LABEL MASTER TABLE
# ============================================================

def ids_to_names(id_list):
    return [genre_name_map.get(int(gid), f"genre_{int(gid)}") for gid in id_list]

large_tracks_audio["genres_all_names"] =
large_tracks_audio["genres_all_ids"].apply(ids_to_names)

large_multilabel_master = pd.DataFrame({
    "track_id": large_tracks_audio.index.astype(int),
    "split": large_tracks_audio[("set", "split")].astype(str).values,
    "subset": large_tracks_audio[("set", "subset")].astype(str).values,
    "genre_top": large_tracks_audio[("track", "genre_top")].astype(str).values,
    "title": large_tracks_audio[("track", "title")].astype(str).values,
    "genres_ids": large_tracks_audio["genres_ids"].apply(lambda x:
",".join(map(str, x)) if len(x) > 0 else ""),
    "genres_all_ids": large_tracks_audio["genres_all_ids"].apply(lambda x:
",".join(map(str, x)) if len(x) > 0 else ""),
    "genres_all_names": large_tracks_audio["genres_all_names"].apply(lambda x: " |
".join(x) if len(x) > 0 else ""),
    "audio_path": large_tracks_audio["audio_path"].astype(str).values,
    "audio_exists": large_tracks_audio["audio_exists"].astype(bool).values
})

print("Large multi-label master table shape:", large_multilabel_master.shape)
display(large_multilabel_master.head(10))
```

```
Large multi-label master table shape: (81574, 10)
```

track_id	track_id	split	subset	genre_top	title	genres_ids	genres_all_ids	genres_al
20	20	training	large	nan	Spiritual Level	76,103	17,10,76,103	Fo Experime So
26	26	training	large	nan	Where is your Love?	76,103	17,10,76,103	Fo Experime So
30	30	training	large	nan	Too Happy	76,103	17,10,76,103	Fo Experime So
46	46	training	large	nan	Yosemite	76,103	17,10,76,103	Fo Experime So
48	48	training	large	nan	Light of Light	76,103	17,10,76,103	Fo Experime So
135	135	training	large	Rock	Father's Day	45,58	58,12,45	Psych-Ro | Lo
137	137	training	large	Experimental	Side A	1,32	32,1,38	Noise Expe
138	138	training	large	Experimental	Side B	1,32	32,1,38	Noise Expe
142	142	training	large	Folk	Punjabi Watery Grave	17	17	
144	144	training	large	Jazz	Wire Up	4	4	

In [12]:

```
# ============================================================
# 12. SAVE OUTPUT TABLES
# ============================================================

os.makedirs("../data/processed", exist_ok=True)

top_level_summary.to_csv(
    "../data/processed/top_level_genre_summary.csv",
    index=False
```

```
)

genre_inventory.reset_index().to_csv(
    "../data/processed/full_genre_inventory.csv",
    index=False
)

candidate_genres.to_csv(
    "../data/processed/modelling_candidate_genres.csv",
    index=False
)

large_multilabel_master.to_csv(
    "../data/processed/large_multilabel_master_table.csv",
    index=False
)

print("Saved:")
print("- ../data/processed/top_level_genre_summary.csv")
print("- ../data/processed/full_genre_inventory.csv")
print("- ../data/processed/modelling_candidate_genres.csv")
print("- ../data/processed/large_multilabel_master_table.csv")
```

```
Saved:
- ../data/processed/top_level_genre_summary.csv
- ../data/processed/full_genre_inventory.csv
- ../data/processed/modelling_candidate_genres.csv
- ../data/processed/large_multilabel_master_table.csv
```

In [13]:

```
# ============================================================
# 13. HIGH-LEVEL PROJECT INTERPRETATION
# ============================================================

print("1. The current notebooks mainly solved single-label top-level genre
classification.")
print("2. The metadata shows that full genre identification is a multi-label
problem, not just a single-label problem.")
print("3. The genres_all field should become the main target for full metadata
genre identification.")
print("4. The genre inventory and candidate tables created here define the usable
label space for the next modelling phase.")
print("5. The large multi-label master table will support future multi-label
structured, audio, and hybrid experiments.")
```

1. The current notebooks mainly solved single-label top-level genre classification.
2. The metadata shows that full genre identification is a multi-label problem, not just a single-label problem.
3. The genres_all field should become the main target for full metadata genre identification.
4. The genre inventory and candidate tables created here define the usable label space for the next modelling phase.
5. The large multi-label master table will support future multi-label structured, audio, and hybrid experiments.